[Nome dell'Università]

[Dipartimento]

[Corso di Laurea in Ingegneria Elettronica / Informatica]

Interfacciamento di un coprocessore video VERA al bus PBI dell'Atari XL/XE tramite ESP32-S3

Progetto della logica di bus, del firmware e dell'emulazione RAMbo 320K con verifica temporale a 1,77 e 1,79 MHz

Relatore:
[Prof. Nome Cognome]

Correlatore:
[Nome Cognome]

Candidato:
Gianluca Renzi
[Matricola]

Anno Accademico 2025/2026

Sommario

Questa tesi descrive il progetto e la verifica del modulo *Vera-Module-RBL-XE*, una scheda di espansione per i computer Atari 8-bit della serie XL/XE che collega al bus del 6502 il coprocessore video VERA del progetto Commander X16. Un microcontrollore ESP32-S3 osserva ogni ciclo del bus, decodifica in software i sedici bit di indirizzo e, a seconda dell'indirizzo, abilita i registri della VERA, fornisce la ROM e la RAM del dispositivo PBI (Parallel Bus Interface) ed emula un'espansione di memoria RAMbo 320K (64 KB di base più 16 banchi da 16 KB).

Il lavoro parte da una prima versione della scheda e del firmware e ne verifica le tempistiche rispetto al bus del 6502 a 1,773 447 MHz (PAL) e 1,789 773 MHz (NTSC). La verifica è analitica: un modello temporale combina il conteggio delle istruzioni nel codice macchina generato con i parametri dei datasheet del 6502, della logica di interfaccia e della FPGA. L'analisi della prima versione ha mostrato che i cicli di lettura serviti dal microcontrollore non rispettavano il tempo di setup del processore, che la logica di interfaccia faceva contesa sul bus dati in quasi ogni lettura e che il firmware conteneva difetti di collocazione in memoria tali da bloccarlo all'avvio.

La tesi presenta le soluzioni adottate: un circuito di abilitazione del traslatore del bus dati realizzato con una sola porta 74LVC20, un ciclo principale che decodifica l'indirizzo durante la fase ϕ_1 con il percorso critico scritto in assembly, il ciclo eseguito su un core dedicato senza interruzioni e la correzione della collocazione in memoria. Secondo il modello il nuovo progetto rispetta il setup del 6502 nel caso peggiore finché la latenza di un accesso della CPU ai registri GPIO non supera circa 14,1 cicli di clock in NTSC e 14,5 in PAL, e nel caso tipico fino a circa 17,0 cicli; i segnali EXTSEL e MPD arrivano intorno al fronte di salita di ϕ_2 invece che 150–300 ns dopo. I risultati non sono ancora confermati da misure: la tesi chiude con il piano di verifica sperimentale, a partire dalla misura di quella latenza.

Indice

Sommario **i**

1 Introduzione **1**
1.1 Contesto . . . 1
1.2 Obiettivi . . . 1
1.3 Metodo . . . 2
1.4 Struttura della tesi . . . 2

2 Il bus dell'Atari XL/XE **3**
2.1 Il processore e il clock di sistema . . . 3
2.2 Il ciclo di bus del 6502 . . . 3
2.2.1 Parametri temporali . . . 4
2.3 Il connettore PBI/ECI e la gestione della memoria . . . 5

3 Architettura hardware del modulo **7**
3.1 Schema a blocchi . . . 7
3.2 Abilitazione del bus dati . . . 8
3.3 Assegnazione dei pin dell'ESP32-S3 . . . 9
3.4 Ritardi della logica di interfaccia . . . 9

4 Protocollo PBI ed espansione RAMbo 320K **11**
4.1 Il dispositivo PBI . . . 11
4.1.1 Selezione tramite `$D1FF` . . . 11
4.1.2 La ROM di gestione . . . 11
4.1.3 Mappa degli indirizzi . . . 11
4.2 L'espansione RAMbo 320K . . . 12
4.2.1 Bank switching tramite PORTB . . . 12
4.2.2 Osservazione passiva della PIA . . . 12

5 Architettura del firmware **15**
5.1 Divisione del lavoro fra i due core . . . 15
5.2 Sequenza di avvio . . . 15
5.3 Collocazione in memoria . . . 15
5.4 Decodifica a tabelle . . . 16
5.5 Il ciclo principale . . . 16
5.5.1 Il percorso critico in assembly . . . 16
5.5.2 Cicli di scrittura . . . 18
5.6 Conteggio dei cicli . . . 18

6 Verifica temporale **19**
6.1 Modello . . . 19
6.1.1 La latenza degli accessi alle periferiche . . . 19
6.2 Campionamento dell'indirizzo . . . 19
6.3 Ciclo di lettura servito dall'ESP32 . . . 20
6.3.1 Lettura dopo una scrittura sui registri . . . 20
6.4 Ciclo di lettura servito dalla VERA . . . 21
6.5 Cicli di scrittura . . . 21
6.5.1 Campionamento del dato . . . 21
6.5.2 Hold del dato verso la VERA . . . 21
6.6 Asserzione di EXTSEL e MPD . . . 22
6.7 Interruzioni . . . 22
6.8 Sintesi grafica . . . 22

7 Dalla prima versione al progetto attuale **25**
7.1 Il ciclo principale iniziale . . . 25
7.2 Problemi individuati e correzioni . . . 25

8 Questioni aperte **29**
8.1 Latenza degli accessi alle periferiche . . . 29
8.2 Uscite dati sempre abilitate . . . 29
8.3 Hold della scrittura verso la VERA . . . 29
8.4 Interruzioni della VERA . . . 29
8.5 Strapping dell'ESP32 . . . 30
8.6 Architettura alternativa: logica di bus in FPGA . . . 30

9 Piano di verifica sperimentale **31**
9.1 Misura della latenza APB . . . 31
9.2 Misure sul bus con analizzatore logico . . . 31
9.3 Prove funzionali . . . 32

10 Conclusioni **33**

A Il modello temporale **37**

Elenco delle figure

2.1 Ciclo di bus del 6502 a 1,79 MHz 4

3.1 Schema a blocchi del modulo 7
3.2 Circuito di abilitazione di U19 8

5.1 Flusso del ciclo principale di `MonitorTask` per un ciclo di bus. 17

6.1 Lettura servita dall'ESP32 20
6.2 Ciclo di scrittura 21
6.3 Hold del dato di scrittura verso la VERA 22
6.4 Margine dei percorsi in funzione della latenza APB (NTSC). 23
6.5 Margine dei percorsi in funzione della latenza APB (PAL). 23

7.1 Lettura nel firmware iniziale 25

Elenco delle tabelle

2.1 Clock del 6502 negli Atari XL/XE e finestra di ϕ_2 alto. 3
2.2 Parametri temporali del 6502 (datasheet Rockwell R6502). 4

3.1 Tabella di verità del circuito di abilitazione. 8
3.2 Assegnazione dei segnali del bus ai GPIO dell'ESP32-S3. 9
3.3 Ritardi della logica di interfaccia usati nel modello. 9

4.1 Indirizzi gestiti dal firmware in modalità PBI. 12
4.2 Valori di PORTB per i 16 banchi (bit 0, 1 e 7 a 1, bit 4 a 0). 12
4.3 Inizializzazione della PIA da parte del sistema operativo e reazione del firmware. 13

5.1 Cicli di CPU lungo i percorsi del ciclo principale (dal disassemblato). 18

6.1 Lettura servita dall'ESP32: istante del dato sul bus e margine. 20
6.2 Esito della verifica temporale del progetto. 22

7.1 Margine del dato in lettura servita dall'ESP32 (NTSC): firmware iniziale e attuale. 26
7.2 Problemi della prima versione e soluzioni adottate. 26

Capitolo 1

Introduzione

1.1 Contesto

I computer Atari 400/800 e le successive serie XL e XE, prodotti fra il 1979 e il 1992, sono basati sul microprocessore MOS 6502 (nella versione «SALLY» 6502C) e su tre circuiti dedicati: ANTIC per la generazione del display tramite DMA, GTIA per il colore e gli sprite, POKEY per l'audio, la tastiera e la comunicazione seriale. Con la serie XL Atari introdusse il connettore *Parallel Bus Interface* (PBI), sostituito nella serie XE dall'*Enhanced Cartridge Interface* (ECI), che porta all'esterno il bus del processore insieme a un piccolo insieme di segnali di controllo: con questi un dispositivo esterno può sostituire porzioni della memoria interna e aggiungere una propria ROM di gestione che il sistema operativo riconosce all'avvio [1].

La VERA (*Versatile Embedded Retro Adapter*) è il coprocessore video e audio del progetto Commander X16, realizzato su una FPGA Lattice iCE40UP5K [12]. Offre 128 KB di VRAM, due layer grafici, 128 sprite, uscita VGA/HDMI e un generatore audio, ed espone al processore 32 registri a 8 bit. Il modulo *Vera-Module-RBL-XE* porta la VERA su un Atari XL/XE attraverso il connettore PBI/ECI.

1.2 Obiettivi

1. Descrivere l'architettura hardware e software del modulo e il ruolo di ogni segnale del bus.

2. Descrivere i servizi che il firmware offre all'Atari oltre all'accesso alla VERA: il dispositivo PBI (ROM e RAM di gestione, selezione tramite `$D1FF`) e l'emulazione dell'espansione RAMbo 320K.

3. Verificare il rispetto delle tempistiche del bus del 6502 alle due frequenze di clock dei sistemi PAL (1,77 MHz) e NTSC (1,79 MHz) e rappresentare con diagrammi temporali i segnali critici.

4. Correggere i problemi individuati nella prima versione, sia nella logica di interfaccia sia nel firmware, e verificare il progetto risultante.

5. Definire il piano di verifica sperimentale.

1.3 Metodo

La verifica è *analitica*: la scheda non è ancora stata provata su un Atari reale. Per il lato ESP32 il punto di partenza è il codice macchina effettivamente generato: il firmware è compilato con la configurazione di produzione (PlatformIO, piattaforma `espressif32@6.6.0`, arduino-esp32 2.0.14 su ESP-IDF 4.4.6, ottimizzazione `-O3`) e la funzione del ciclo principale è disassemblata per contare le istruzioni lungo ogni percorso. Per il lato Atari e per la logica di interfaccia si usano i parametri dei datasheet. Le grandezze che i documenti disponibili non specificano, in particolare la latenza di accesso della CPU ai registri GPIO dell'ESP32-S3, sono trattate come parametri e i risultati sono presentati in funzione di essi, così da distinguere le conclusioni che valgono in ogni caso da quelle che dipendono da una misura ancora da fare.

Tutti i numeri delle tabelle e dei grafici sono prodotti da un unico script, `timing_model.py`, allegato alla tesi (appendice A). I riferimenti al codice indicano i commit del repository del progetto: `f311311` per il firmware iniziale, `82c429f` e successivi per il firmware attuale.

1.4 Struttura della tesi

Il capitolo 2 descrive il bus dell'Atari XL/XE e i parametri temporali di riferimento. Il capitolo 3 presenta l'hardware del modulo, compreso il circuito di abilitazione del bus dati. Il capitolo 4 descrive il protocollo PBI e la RAMbo 320K. Il capitolo 5 descrive il firmware. Il capitolo 6 contiene la verifica temporale del progetto. Il capitolo 7 ripercorre l'analisi della prima versione e le correzioni che ne sono seguite. Il capitolo 8 raccoglie le questioni ancora aperte, il capitolo 9 il piano di verifica sperimentale e il capitolo 10 le conclusioni.

Capitolo 2

Il bus dell'Atari XL/XE

2.1 Il processore e il clock di sistema

Gli Atari XL/XE usano il 6502C «SALLY», una versione del 6502 NMOS con la logica di HALT integrata: ANTIC usa HALT per sospendere il processore durante i propri cicli di DMA [1]. Il clock del processore deriva dall'oscillatore del colore: nei sistemi NTSC il 6502 gira a metà della frequenza della sottoportante colore, nei sistemi PAL ai due quinti della sottoportante [1, §2.2]. Ne risultano le grandezze della tabella 2.1.

Tabella 2.1: Clock del 6502 negli Atari XL/XE e finestra di ϕ_2 alto.

	PAL	**NTSC**
Frequenza $f_{\phi 2}$	1,773 447 MHz	1,789 773 MHz
Periodo $T = 1/f_{\phi 2}$	563,9 ns	558,7 ns
ϕ_2 alto, $T_H = T/2$	281,9 ns	279,4 ns
Cicli per riga di scansione	114	114
Cicli per quadro	35 568	29 868
Cicli ESP32-S3 a 240 MHz in T_H	67,7	67,0

Il periodo PAL è più lungo di circa 5 ns e la finestra di ϕ_2 alto di circa 2,6 ns: il sistema NTSC, più veloce, è il caso peggiore e quello di riferimento. Il clock ϕ_0 generato da ANTIC si ottiene dividendo per due il clock del colore ed è nominalmente simmetrico; il ϕ_2 in uscita dal processore, presente sul connettore PBI, può discostarsi dal 50 % di qualche nanosecondo. Nell'analisi del caso peggiore si assume una riduzione di 10 ns della fase alta, da sostituire con il valore misurato.

2.2 Il ciclo di bus del 6502

Il 6502 esegue un accesso alla memoria per ciclo di clock, senza stati di attesa. Il ciclo si divide in due fasi:

ϕ_1 **(ϕ_2 basso)** il processore porta sul bus il nuovo indirizzo e il livello di $R/\overline{W}$; il bus dati non è usato;

ϕ_2 **alto** la memoria o la periferica indirizzata risponde. In lettura il processore campiona il bus dati sul fronte di discesa di ϕ_2; in scrittura il processore pilota il bus dati.

Negli Atari la linea RDY è riservata ad ANTIC: un dispositivo esterno non può allungare un ciclo, e un dispositivo che non risponde in tempo produce un dato errato.

2.2.1 Parametri temporali

Il datasheet di SALLY non è pubblico. Poiché il processore funziona a 1,79 MHz, i suoi parametri devono essere almeno quelli della versione a 2 MHz del 6502; la tabella 2.2 riporta i valori del datasheet Rockwell R6502 [2].

Tabella 2.2: Parametri temporali del 6502 (datasheet Rockwell R6502).

Simbolo	Parametro	1 MHz	2 MHz	
t_{ADS}	indirizzo valido dopo l'inizio di ϕ_1	225	140	max
t_{RWS}	$R/\overline{W}$ valido dopo l'inizio di ϕ_1	225	140	max
t_{ACC}	tempo di accesso disponibile alla memoria	650	310	min
t_{DSU}	setup del dato in lettura prima di $\phi_2 \downarrow$	100	50	min
t_{HR}	hold del dato in lettura dopo $\phi_2 \downarrow$	10	10	min
t_{MDS}	dato di scrittura valido dopo $\phi_2 \uparrow$	175	100	max
t_{HW}	hold del dato di scrittura dopo $\phi_2 \downarrow$	30	30	min
t_{HA}	hold dell'indirizzo dopo $\phi_2 \downarrow$	30	30	min

Valori in nanosecondi.

Da questi valori derivano i limiti che dominano l'analisi:

- in un ciclo di lettura il dato deve essere stabile sul bus entro

$$t_{\text{dato}} \leq T_H - t_{DSU} \tag{2.1}$$

  dal fronte di salita di ϕ_2: 229 ns in NTSC e 232 ns in PAL con ϕ_2 simmetrico, 219 ns e 222 ns con la riduzione di 10 ns della fase alta;

- in un ciclo di scrittura il dato del processore è garantito solo fra $t_{MDS} = 100$ ns dopo la salita di ϕ_2 e $t_{HW} = 30$ ns dopo la discesa;

- l'indirizzo è stabile già durante ϕ_1: al più 140 ns dopo l'inizio di ϕ_1, cioè circa 140 ns *prima* della salita di ϕ_2. Il firmware attuale sfrutta questo intervallo per decodificare l'indirizzo in anticipo.

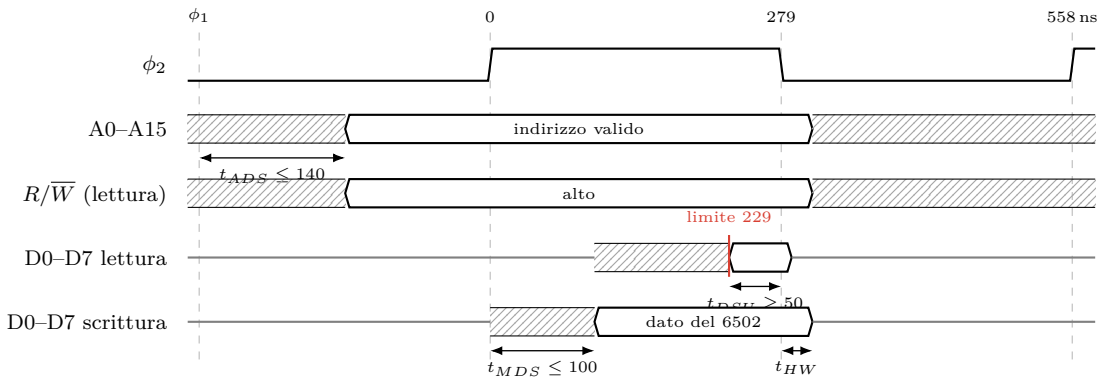


Figura 2.1: Ciclo di bus del 6502 a 1,79 MHz (NTSC) in scala, con i parametri della versione a 2 MHz. Le zone tratteggiate sono valori non garantiti; la riga grigia indica il bus in alta impedenza.

2.3 Il connettore PBI/ECI e la gestione della memoria

Il connettore PBI degli XL porta il bus indirizzi e dati, ϕ_2, $R/\overline{W}$, IRQ, RESET, HALT, REFRESH e tre segnali che un dispositivo esterno usa per sovrapporsi alla memoria interna [1, cap. 10]:

EXTSEL (attivo basso) disabilita la RAM interna nel ciclo corrente;

MPD (*Math Pack Disable*, attivo basso) disabilita la ROM interna del Math Pack a `$D800`–`$DFFF`, la finestra della ROM dei dispositivi PBI;

D1XX decodifica della pagina `$D1xx` riservata ai dispositivi PBI.

La logica di mappatura della memoria (MMU) decide il dispositivo interno da abilitare in base all'indirizzo e allo stato di PORTB [1, §3.4]. Un dispositivo esterno può sovrapporsi alla RAM e, tramite MPD, alla sola ROM del Math Pack. I dispositivi PBI commerciali decodificano l'indirizzo con logica combinatoria, per cui EXTSEL e MPD sono validi già durante ϕ_1; la documentazione disponibile non fissa un limite esplicito rispetto alla generazione di RAS/CAS della RAM interna, e il punto è discusso nella sezione 6.6.

Capitolo 3

Architettura hardware del modulo

3.1 Schema a blocchi

La figura 3.1 riassume l'hardware [14, 15]. Il bus Atari è a 5 V TTL, mentre ESP32-S3 e FPGA sono a 3,3 V: i segnali attraversano traslatori di livello 74LVC4245 (otto canali, direzione impostata dal pin DIR, abilitazione dal pin $\overline{\text{OE}}$, alimentazioni separate per i due lati: VCCA a 5 V sul pin 1, VCCB a 3,3 V sui pin 23 e 24).

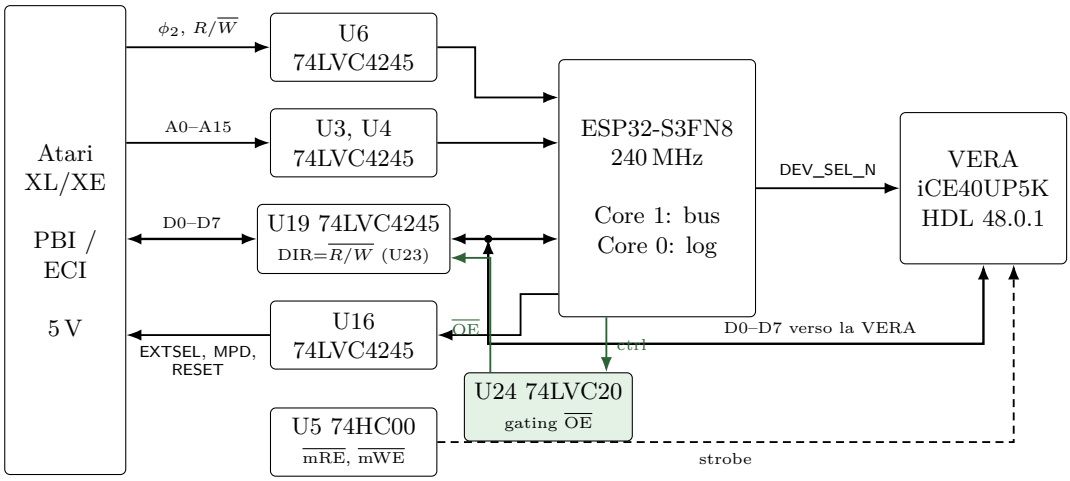


Figura 3.1: Schema a blocchi del modulo Vera-Module-RBL-XE con il circuito di abilitazione di U19 (in verde), che riceve dall'ESP32 i tre segnali di controllo e da U6 ϕ_2 e $R/\overline{W}$. Riferimenti dei componenti dalla netlist del 2026-10-23.

U6 traslatore Atari→scheda per ϕ_2, $R/\overline{W}$, D1XX, S4, S5, CCTL, REFRESH;

U3, U4 traslatori del bus indirizzi (A8–A15 e A0–A7);

U19 traslatore bidirezionale del bus dati: DIR è $\overline{R/W}$ (da U23, 74HCT04), $\overline{\text{OE}}$ viene dal circuito di abilitazione U24 (sezione 3.2);

U16 traslatore scheda→Atari per EXTSEL, MPD, RESET, HALT, RD4, RD5, IRQ;

U5 porte NAND che generano gli strobe della VERA: $\overline{\text{mRE}} = \overline{\phi_2 \cdot R/W}$, $\overline{\text{mWE}} = \overline{\phi_2 \cdot \overline{R/W}}$;

ESP32-S3FN8 microcontrollore dual core Xtensa LX7 a 240 MHz con 8 MB di flash integrata e 512 KB di SRAM [4]; DEV_SEL_N è il chip select della VERA;

VERA FPGA iCE40UP5K con il firmware VERA 48.0.1, configurata da flash SPI all'uscita dal reset (CRESET); il pin CDONE ne segnala il completamento.

3.2 Abilitazione del bus dati

U19 collega il bus dati dell'Atari al bus dati della scheda, condiviso da ESP32 e VERA. Nella prima versione il suo $\overline{\text{OE}}$ era $\overline{\phi_2}$: U19 era attivo in *ogni* ciclo con ϕ_2 alto. In una lettura della RAM o della ROM interne, con DIR rivolto verso l'Atari, U19 pilotava quindi il bus dati dell'Atari con i livelli indefiniti del lato scheda, in contesa con la memoria interna. Il circuito di abilitazione attiva U19 solo quando ϕ_2 è alto *e* il ciclo è una scrittura (la scheda deve osservare tutte le scritture) oppure la scheda sta rispondendo, cioè quando il firmware ha asserito almeno uno fra DEV_SEL_N, EXTSEL_N e MPD:

$$\begin{aligned}\overline{\text{OE}} &= \overline{\phi_2 \cdot (\overline{R/W} + \overline{\text{DEV_SEL_N}} + \overline{\text{EXTSEL_N}} + \overline{\text{MPD}})} \\ &= \text{NAND}(\phi_2,\ \text{NAND}_4(R/W,\ \text{DEV_SEL_N},\ \text{EXTSEL_N},\ \text{MPD})).\end{aligned} \tag{3.1}$$

La seconda forma si realizza con un solo 74LVC20 (due NAND a quattro ingressi), la seconda porta usata come NAND a due ingressi con gli ingressi liberi a 3,3 V (figura 3.2, tabella 3.1). I segnali sono presi dal lato a 3,3 V, dove esiste DEV_SEL_N; il pin $\overline{\text{OE}}$ di U19 è riferito a VCCA con soglia TTL ($V_{IH} \geq 2{,}0$ V) e l'uscita LVC lo pilota direttamente. L'uscita di U23 che prima pilotava $\overline{\text{OE}}$ resta scollegata.

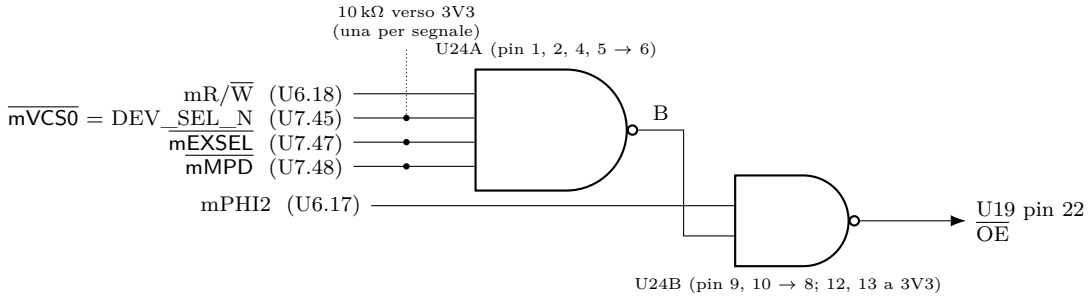


Figura 3.2: Circuito di abilitazione di U19 con un 74LVC20 (U24). I tre segnali di controllo hanno ciascuno una resistenza di pull-up da 10 kΩ verso 3V3, perché durante il reset dell'ESP32 i pin che li pilotano non hanno un livello garantito.

Tabella 3.1: Tabella di verità del circuito di abilitazione.

ϕ_2	$R/\overline{W}$	controlli	B	$\overline{\text{OE}}$	U19
0	x	x	x	1	spento (ϕ_1)
1	0	x	1	0	acceso, Atari → scheda
1	1	tutti a 1	0	1	spento: risponde la memoria interna
1	1	almeno uno a 0	1	0	acceso, scheda → Atari

Una lettura di `$D1FF` non asserisce alcun segnale e lascia U19 spento: il bus resta non pilotato e le resistenze di pull-up degli XL lo portano a `$FF`. È voluto: l'identificazione delle interruzioni tramite `$D1FF` non è realizzabile con un traslatore a otto canali comuni, e le interruzioni della VERA sono gestite via software [15, §2.4].

3.3 Assegnazione dei pin dell'ESP32-S3

La tabella 3.2 riporta l'assegnazione [14]. Restano liberi solo GPIO 47 e 48, troppo pochi per usare i segnali di decodifica hardware del connettore: la decodifica dell'indirizzo è interamente software. GPIO 3 non è usato dal firmware; essendo un pin di strapping senza resistenze interne, sulla scheda ha una resistenza esterna che ne fissa il livello all'avvio.

Tabella 3.2: Assegnazione dei segnali del bus ai GPIO dell'ESP32-S3.

Segnale	GPIO	Lettura	Note
ϕ_2	1	Dedicated GPIO ingr. 0	anche `GPIO.in` bit 1
$R/\overline{W}$	2	Dedicated GPIO ingr. 1	
D0–D7	4–11	`GPIO.in` 4–11	bidirezionale
A0–A7	12–19	`GPIO.in` 12–19	GPIO 19 = USB D−
A8, A9	20, 21	`GPIO.in` 20–21	GPIO 20 = USB D+
A10–A15	33–36, 45, 46	Dedicated GPIO ingr. 2–7	45, 46: strapping
ARESET, CRESET	37, 38	uscite	reset di Atari e VERA
CDONE	39	`GPIO.in1`	VERA configurata
DEV_SEL_N	40	Dedicated GPIO usc. 0	chip select VERA
EXTSEL_N	41	Dedicated GPIO usc. 1	verso il bus via U16
MPD	42	Dedicated GPIO usc. 2	verso il bus via U16
UART0 TX, RX	43, 44		log di debug

Il modulo *Dedicated GPIO* dell'ESP32-S3 collega fino a otto ingressi e otto uscite a istruzioni della CPU (`ee.get_gpio_in`, `ee.wr_mask_gpio_out`) eseguite in un ciclo di clock, senza passare dal bus delle periferiche [5, 6]. Il firmware usa tutti gli otto canali di ingresso per ϕ_2, $R/\overline{W}$ e A10–A15: la matrice GPIO collega gli stessi pin anche ai registri ordinari, quindi l'assegnazione non richiede modifiche alla scheda. A0–A9 e i dati restano in `GPIO.in`, un registro del bus delle periferiche: è l'unico accesso di questo tipo necessario per campionare l'indirizzo.

3.4 Ritardi della logica di interfaccia

La tabella 3.3 riporta i ritardi usati nel modello. Per il 74HC00 alimentato a 3,3 V il datasheet fornisce solo i valori a 2 e 4,5 V e quelli in tabella sono interpolati; il valore per la FPGA è una stima del percorso pad–logica–pad.

Tabella 3.3: Ritardi della logica di interfaccia usati nel modello.

Componente	Percorso	min	tip	max
74LVC4245A [7]	propagazione 5 ↔ 3,3 V	1,0	4,0	6,5
74LVC4245A	abilitazione uscita	1,0	5,0	9,0
74LVC4245A	disabilitazione uscita	1,0	4,0	8,0
74LVC20 [8]	propagazione a 3,3 V, per porta	1,0	2,5	4,5
74HCT04 [9]	propagazione a 5 V	4	10	24
74HC00 [10]	propagazione a 3,3 V (interp.)	5	12	30
iCE40UP5K [11]	pad → logica → pad (stima)	6	10	15
ESP32-S3	sincronizzatore di ingresso GPIO	0	12,5	25
ESP32-S3	uscita GPIO verso il pad	2	4	6

Valori in nanosecondi.

Capitolo 4

Protocollo PBI ed espansione RAMbo 320K

4.1 Il dispositivo PBI

4.1.1 Selezione tramite `$D1FF`

Più dispositivi PBI possono essere collegati in catena. Il registro `$D1FF` (PDVS in scrittura) seleziona il dispositivo attivo: ogni dispositivo è associato a un bit e scrivere un valore con il proprio bit a 1 lo seleziona, scrivere `$00` deseleziona tutti [1, §10.2]. Il registro non esiste nel computer: ogni dispositivo lo implementa catturando le scritture all'indirizzo. Il modulo usa il bit 7 e considera solo il proprio bit. Quando è selezionato la sua ROM appare a `$D800`–`$DFFF` al posto del Math Pack (MPD) e la pagina `$D1xx` e l'area `$D600`–`$D7FF` appartengono a lui (EXTSEL); quando non è selezionato non risponde a nessuno di questi indirizzi, così da poter convivere con altri dispositivi PBI. I registri della VERA (`$D100`–`$D11F`) sono quindi accessibili solo con la scheda selezionata.

4.1.2 La ROM di gestione

All'avvio il sistema operativo XL/XE seleziona a turno ogni bit di `$D1FF`, verifica l'intestazione della ROM e, se valida, chiama la routine di inizializzazione del dispositivo. La ROM del modulo (`vera_pbi_handler.s`, 2 KB assemblati con `ca65` all'indirizzo `$D800`) espone `$D803` = `$80` (il bit del dispositivo), `$D80B` = `$91` (firma), `$D80C` = `'V'` e i vettori `$D805` (SIO) e `$D808` (IRQ). Ogni lettura di questi byte è un ciclo di lettura servito dall'ESP32: la correttezza temporale di questi cicli (sezione 6.3) è la condizione perché la scheda venga riconosciuta. Il firmware serve l'immagine generata dal makefile del driver a ogni compilazione (`vera_pbi_handler.h`); una verifica in compilazione ne controlla la dimensione di 2 KB.

4.1.3 Mappa degli indirizzi

In modalità CCTL (cartuccia) il latch è a `$D5FF` e i registri della VERA occupano `$D500`–`$D5FE`; non ci sono ROM e RAM PBI.

Tabella 4.1: Indirizzi gestiti dal firmware in modalità PBI.

Intervallo	Condizione	Segnali	Dato
`$D100`–`$D11F`	scheda selezionata	EXTSEL, DEV_SEL_N	VERA
`$D120`–`$D1FE`	scheda selezionata	EXTSEL	non pilotato
`$D1FF`	scrittura	—	latch di selezione
`$D300`–`$D3FF`	scrittura	—	osservazione PIA
`$D600`–`$D7FF`	scheda selezionata	EXTSEL	ESP32, RAM 512 B
`$D800`–`$DFFF`	scheda selezionata	MPD	ESP32, ROM 2 KB
`$4000`–`$7FFF`	RAMbo attiva	EXTSEL	ESP32, banco 16 KB

4.2 L'espansione RAMbo 320K

4.2.1 Bank switching tramite PORTB

Negli XL/XE la porta B della PIA 6520 (`$D301`, PORTB) controlla la mappa della memoria: il bit 0 abilita la ROM del sistema operativo, il bit 1 il BASIC, il bit 7 la ROM di autotest a `$5000`–`$57FF` [1, §3.4]. L'espansione «320K Rambo» usa i bit 2, 3, 5 e 6 per selezionare uno di 16 banchi da 16 KB visti nella finestra `$4000`–`$7FFF` e il bit 4 per abilitarla: $64 + 16 \times 16 = 320$ KB. CPU e ANTIC vedono sempre lo stesso banco [1, §3.5, §11]. Il banco è

$$\text{banco} = ((\text{PORTB} \gg 2)\ \&\ 3)\ |\ ((\text{PORTB} \gg 3)\ \&\ 0\text{x}0\text{C}). \tag{4.1}$$

L'emulazione è sempre attiva: la scheda non ha un segnale di abilitazione. Poiché l'ESP32 non distingue i cicli della CPU da quelli di DMA di ANTIC (non osserva HALT), risponde a entrambi con lo stesso banco, che è il comportamento della RAMbo 320K. Quando la ROM del sistema e la ROM di autotest sono entrambe abilitate (bit 0 = 1, bit 7 = 0), l'autotest ha la precedenza su `$5000`–`$57FF` e la scheda lascia quelle pagine alla ROM interna.

Tabella 4.2: Valori di PORTB per i 16 banchi (bit 0, 1 e 7 a 1, bit 4 a 0).

Banco	PORTB	Banco	PORTB	Banco	PORTB	Banco	PORTB
0	`$83`	4	`$A3`	8	`$C3`	12	`$E3`
1	`$87`	5	`$A7`	9	`$C7`	13	`$E7`
2	`$8B`	6	`$AB`	10	`$CB`	14	`$EB`
3	`$8F`	7	`$AF`	11	`$CF`	15	`$EF`

4.2.2 Osservazione passiva della PIA

La PIA fisica resta sulla scheda madre e risponde normalmente: l'ESP32 non la emula ma ne *osserva* le scritture. A `$D301` la PIA espone due registri, il registro di uscita e il registro di direzione (DDR), selezionati dal bit 2 del registro di controllo PBCTL (`$D303`); sul bus le due scritture sono indistinguibili. Il firmware tiene una copia di PBCTL, del registro di uscita e del DDR e calcola i livelli effettivi dei pin:

$$\text{pin} = \text{uscita}\ |\ \overline{\text{DDR}}, \tag{4.2}$$

perché i bit configurati come ingressi sono portati a 1 dalle resistenze di pull-up. Dopo il reset della PIA entrambi i registri valgono 0 e i pin leggono `$FF` (RAMbo disabilitata). La PIA decodifica solo A0–A1, quindi le scritture valgono anche agli indirizzi speculari in `$D300`–`$D3FF`. La tabella 4.3 mostra la sequenza tipica del sistema operativo.

Tabella 4.3: Inizializzazione della PIA da parte del sistema operativo e reazione del firmware.

Passo	**Scrittura**	**PIA**	**Firmware**
1	`$00` → `$D303`	DDR selezionato a `$D301`	copia di PBCTL
2	`$FF` → `$D301`	tutti i pin in uscita	copia del DDR
3	`$34` → `$D303`	registro di uscita a `$D301`	copia di PBCTL
4	`$NN` → `$D301`	nuova mappa di memoria	copia del registro, nuovo stato

Capitolo 5

Architettura del firmware

5.1 Divisione del lavoro fra i due core

Il firmware è un'applicazione Arduino su ESP-IDF e FreeRTOS. Il core 1 esegue solo il ciclo principale (`MonitorTask`) con le interruzioni mascherate: nessun tick di FreeRTOS, nessuna interruzione di periferica può sottrargli cicli. Il core 0 esegue `LogTask`, che apre la seriale (l'interruzione della UART viene allocata sul core che la inizializza) e stampa gli eventi prodotti dal ciclo principale, e continua ad aggiornare il contatore dei tick. Gli eventi passano attraverso un buffer circolare senza lock con un solo produttore e un solo consumatore: l'inserimento è un paio di store, il timestamp lo aggiunge `LogTask` quando stampa. Il watchdog delle interruzioni di ESP-IDF controlla che ogni core esegua il proprio tick: poiché il core 1 non lo esegue più, il ciclo principale lo disattiva prima di mascherare le interruzioni.

5.2 Sequenza di avvio

1. `setup()` porta a livello basso ARESET e CRESET: Atari e VERA restano in reset per tutta l'inizializzazione;

2. crea `LogTask` sul core 0, costruisce le tabelle di decodifica e configura i GPIO;

3. rilascia CRESET, attende CDONE (timeout 5 s) e altri 1 ms per il reset interno della VERA;

4. crea `MonitorTask` sul core 1 con la priorità massima. Il task crea i canali Dedicated GPIO, ne verifica la posizione, disattiva il watchdog delle interruzioni, maschera le interruzioni e rilascia ARESET: da questo momento il bus è attivo.

5.3 Collocazione in memoria

Sull'ESP32-S3 la SRAM interna è visibile sia dal bus istruzioni sia dal bus dati, ma solo gli indirizzi del bus dati sono accessibili a byte [6]. ROM PBI, RAM PBI, array della RAMbo (256 KB) e tabelle di decodifica sono quindi in DRAM ordinaria: la RAMbo come `.bss`, azzerata all'avvio senza costo in flash. Il firmware occupa 287 KB dei 320 KB della regione dati. Il codice del ciclo principale è in IRAM, così che non subisca mancate in cache, e non chiama funzioni.

5.4 Decodifica a tabelle

Tutto ciò che dipende dalla pagina (A8–A15) e dallo stato della scheda è precalcolato (`bus_decode.h`). La pagina si ottiene dai due campioni senza riordinare bit:

```
/* din = ee.get_gpio_in: bit 0 = PHI2, bit 1 = R/W, bit 2..7 = A10..A15 */
/* lo  = GPIO.in:        bit 12..21 = A0..A9                            */
uint32_t page = ((lo >> 20) & 0x03u) | (din & 0xFCu);   /* A15..A8 */
```

Listing 5.1: Numero di pagina dai due campioni (`bus_decode.h`).

Ogni voce delle tabelle contiene nei bit 0–2 i livelli di DEV_SEL_N, EXTSEL_N e MPD, pronti per `ee.wr_mask_gpio_out`, e nei bit 3–7 le azioni: `CT_SLOW` (pagina `$D1xx`/`$D5xx`, che dipende anche da A0–A7), `CT_DRIVE` (pilotare il bus in lettura), `CT_RAMBO` (aggiungere l'offset del banco), `CT_WR_MEM` (memorizzare il dato in scrittura), `CT_WR_REG` (latch o osservazione della PIA). Le tabelle sono otto, una per ogni combinazione di scheda selezionata, RAMbo attiva e ROM di autotest visibile; il cambio di stato cambia solo un puntatore. La pagina `CT_SLOW` ha una seconda tabella indicizzata da A0–A7, una per scheda selezionata e una per scheda non selezionata, e una terza tabella da 256 voci dà i bit di stato per ogni valore di PORTB. La logica non dipende da ESP-IDF ed è verificata sul PC contro una versione a confronti di intervalli per tutti i 65 536 indirizzi e tutti gli stati (16 794 373 confronti, nessuna differenza).

5.5 Il ciclo principale

Il ciclo principale è organizzato attorno al fatto che l'indirizzo è stabile prima della salita di ϕ_2 (figura 5.1):

1. alla discesa di ϕ_2 chiude il ciclo precedente: rilascia il bus dati e i segnali di controllo ed esegue l'eventuale scrittura in memoria rimasta in coda dal ciclo di scrittura precedente;
2. attende 150 ns dalla discesa di ϕ_2 (t_{ADS} più il ritardo dei traslatori, meno il percorso di rilevamento più breve), contati con il contatore di cicli `CCOUNT`;
3. campiona l'indirizzo con una lettura dei canali Dedicated GPIO e una lettura di `GPIO.in`, legge la voce della tabella e asserisce i segnali di controllo;
4. nei cicli di lettura di una pagina servita dall'ESP32 legge il byte e lo porta sui pin dati: U19 lo fa passare verso l'Atari solo con ϕ_2 alto;
5. nei cicli di scrittura campiona il dato 120 ns dopo la salita di ϕ_2, mette in coda la scrittura in memoria per il ϕ_1 successivo e applica subito gli effetti sui registri (latch, PIA);
6. attende la discesa di ϕ_2.

5.5.1 Il percorso critico in assembly

Dalla fine dell'attesa al dato sui pin il codice è un blocco di assembly inline (listato 5.2). Tutti gli operandi sono in registro e le costanti sono caricate prima dell'attesa; il blocco non contiene accessi allo stack né barriere `memw`. Le due scritture del dato (`GPIO.out` e `GPIO.enable_w1ts`) sono emesse una dopo l'altra: gli store Xtensa restano in ordine di programma, quindi il valore raggiunge `GPIO.out` prima che le uscite siano abilitate, ma

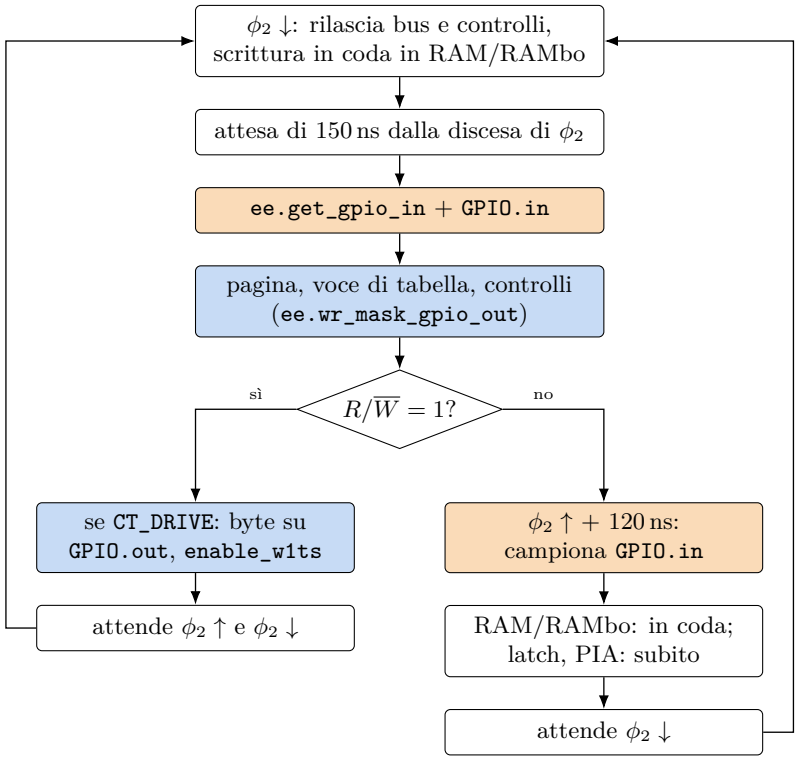


Figura 5.1: Flusso del ciclo principale di `MonitorTask` per un ciclo di bus.

la CPU non aspetta il completamento di ciascuna. La pagina `CT_SLOW` esce dal percorso principale per leggere la seconda tabella e vi rientra prima dell'asserzione dei controlli; non pilota mai il bus.

```
     movi    c, 0xFC              ; maschera A10-A15
     movi    t2, 7                ; maschera dei controlli
1:   rsr.ccount t1                ; attesa di assestamento
     sub     t1, t1, ts
     bltz    t1, 1b
     ee.get_gpio_in din           ; PHI2, R/W, A10-A15
     l32i    lo, gpio, 60         ; GPIO.in: A0-A9, D0-D7
     and     page, din, c
     extui   t1, lo, 20, 2        ; A8, A9
     or      page, page, t1
     add     t1, ctrl, page
     l8ui    c, t1, 0             ; voce della tabella
     extui   off, lo, 12, 8       ; A0-A7
     bbsi    c, 3, 5f             ; CT_SLOW
3:   ee.wr_mask_gpio_out c, t2    ; DEV_SEL_N / EXTSEL_N / MPD
     bbci    din, 1, 4f           ; scrittura
     bbci    c, 4, 4f             ; non CT_DRIVE
     addx4   t1, page, rdb
     l32i    t1, t1, 0            ; bus_rd_base[page]
     extui   t2, c, 5, 1          ; CT_RAMBO
     neg     t2, t2
     and     t2, t2, roff         ; offset del banco solo per la RAMbo
     add     t1, t1, off
     add     t1, t1, t2
     l8ui    t1, t1, 0            ; byte da servire
     movi    t2, 0xFF0
     slli    t1, t1, 4
```

```
28      s32i    t1, gpio, 4          ; GPIO.out
29      s32i    t2, gpio, 36         ; GPIO.enable_w1ts
30  4:  j       6f
31  5:  add     t1, slow, off        ; pagina $D1xx / $D5xx
32      l8ui    c, t1, 0
33      j       3b
34  6:
```

Listing 5.2: Percorso critico del ciclo principale (assembly inline in `main.cpp`).

5.5.2 Cicli di scrittura

Il dato di una scrittura è valido sul bus da t_{MDS} dopo la salita di ϕ_2 fino a t_{HW} dopo la discesa. Il firmware lo campiona a tempo fisso, 120 ns dopo aver rilevato la salita, e rileva la discesa con i canali Dedicated GPIO. Gli effetti della scrittura sono divisi secondo la loro durata:

- la scrittura in RAM PBI o RAMbo (circa 15 cicli) è messa in coda ed eseguita nel ϕ_1 successivo, dove con il resto del lavoro di ϕ_1 resta dentro l'attesa di assestamento;
- il latch di selezione e la copia di PORTB (fino a circa 45 cicli, perché cambiano lo stato e le tabelle) sono applicati subito, nella parte restante di ϕ_2, e possono proseguire nel ϕ_1 successivo.

Nel secondo caso il ciclo rileva la discesa di ϕ_2 in ritardo. Per non spostare di altrettanto il campionamento successivo, l'attesa di assestamento parte dal minore fra l'istante rilevato e quello previsto, cioè la salita di ϕ_2 più la durata massima della fase alta (292 ns: il valore PAL con 10 ns di tolleranza sul duty cycle). Il log del cambio di latch è differito al successivo ciclo di lettura, durante l'attesa della discesa di ϕ_2.

5.6 Conteggio dei cicli

La tabella 5.1 riassume i percorsi contati sul disassemblato. Le istruzioni sono contate a un ciclo, più due cicli per ogni salto preso; gli accessi alle periferiche sono indicati separatamente con la latenza L_{APB} (sezione 6.1.1).

Tabella 5.1: Cicli di CPU lungo i percorsi del ciclo principale (dal disassemblato).

Tratto	Cicli	Accessi APB
attesa dei fronti di ϕ_2 (per campione)	3	—
ϕ_1: chiusura del ciclo e scrittura in coda	≤ 32	1 scrittura (posted)
attesa di assestamento (granularità)	5	—
campionamento e decodifica fino ai controlli	10	1 lettura
idem per `$D1xx`/`$D5xx`	17	1 lettura
dai controlli al dato sui pin	14	2 scritture (posted)

Capitolo 6

Verifica temporale

6.1 Modello

Tutti i tempi sono misurati dal fronte di ϕ_2 sul connettore dell'Atari. Un percorso è la somma di ritardi hardware (tabella 3.3) e di tempi di esecuzione (tabella 5.1, ciclo di CPU $t_c = 4{,}167\,\text{ns}$). Per ogni percorso si calcola il *margine*: il limite meno il tempo calcolato. Si considerano due casi:

tipico ritardi tipici, ϕ_2 simmetrico, attese a metà della loro granularità;

peggiore ritardi massimi, sincronizzatore di ingresso a due cicli di APB, fronti rilevati alla fine dell'iterazione di attesa, fase alta di ϕ_2 ridotta di 10 ns.

Le scritture del dato sui pin sono contate come se il bus delle periferiche le eseguisse una dopo l'altra, l'ipotesi meno favorevole.

6.1.1 La latenza degli accessi alle periferiche

La CPU raggiunge i registri GPIO ordinari attraverso il bus delle periferiche, che funziona a 80 MHz. La latenza di un accesso non è specificata nei documenti di Espressif disponibili. Nel modello è un parametro L_{APB} con tre valori di riferimento:

Scenario	L_{APB} **(cicli)**	L_{APB} **(ns)**
ottimistico (valore del progetto)	6	25
nominale	12	50
pessimistico	20	83

La sezione 9.1 descrive come misurarla con un programma di prova.

6.2 Campionamento dell'indirizzo

L'indirizzo è garantito stabile al pin dell'ESP32 al più $t_{ADS} + t_{U3,U4} = 146{,}5\,\text{ns}$ dopo la discesa di ϕ_2. Il firmware conta l'attesa dall'istante in cui rileva la discesa, che arriva almeno 1 ns dopo il fronte (U6), e la sincronizzazione degli ingressi agisce allo stesso modo su ϕ_2 e sugli indirizzi: un'attesa di 150 ns (36 cicli) garantisce quindi il campionamento di un indirizzo stabile. Nel caso peggiore il campionamento cade $-70\,\text{ns}$ dalla salita di ϕ_2 (cioè prima della salita) in NTSC, nel caso tipico $-92\,\text{ns}$.

6.3 Ciclo di lettura servito dall'ESP32

È il caso delle letture della ROM PBI, della RAM PBI e della finestra RAMbo. Il dato arriva sul bus Atari quando sono vere entrambe le condizioni: il byte è sui pin dell'ESP32 e U19 è abilitato. Il primo percorso è

$$t_{\text{pin}} = t_{\text{camp}} + (10 + L_{APB})\, t_c + t_{\text{pad}} + (14 + 2\,L_{APB})\, t_c, \tag{6.1}$$

il secondo è l'asserzione del segnale di controllo più due porte del 74LVC20 e il tempo di abilitazione del 4245. Il secondo è sempre più breve del primo di almeno 60 ns: il circuito di abilitazione non è sul percorso critico. La figura 6.1 mostra il ciclo nel caso peggiore con latenza nominale, la tabella 6.1 tutti gli scenari.

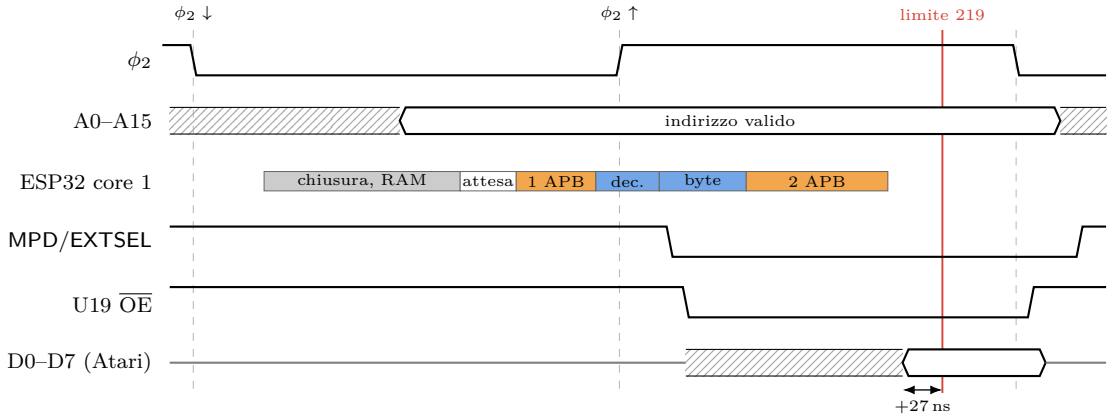


Figura 6.1: Ciclo di lettura della ROM PBI, caso peggiore in NTSC con $L_{APB} = 12$ cicli. L'indirizzo è campionato −70 ns prima della salita di ϕ_2; MPD arriva sul bus 34 ns dopo la salita, U19 si abilita a 45 ns e il dato è sul bus a 192 ns, 27 ns prima del limite.

Tabella 6.1: Lettura servita dall'ESP32: istante del dato sul bus e margine.

	NTSC				PAL			
	tipico		peggiore		tipico		peggiore	
Scenario	t	marg.	t	marg.	t	marg.	t	marg.
ottimistico	91	138	117	102	88	144	115	107
nominale	166	63	192	27	163	69	190	32
pessimistico	266	−37	292	−73	263	−31	290	−68

Tempi dalla salita di ϕ_2 e margini in nanosecondi.

Risultato. Il dato rispetta il setup del 6502 nel caso peggiore finché $L_{APB} \leq 14{,}1$ cicli in NTSC e 14,5 in PAL; nel caso tipico fino a circa 17,0 cicli. Nello scenario pessimistico ($L_{APB} = 20$) il limite non è rispettato: la conformità dipende quindi dalla misura di L_{APB}.

6.3.1 Lettura dopo una scrittura sui registri

Dopo una scrittura del latch o di PORTB gli effetti possono superare la fine di ϕ_2 e l'attesa di assestamento parte dalla discesa prevista invece che da quella rilevata. Nel caso peggiore il campionamento successivo slitta al più di 23 ns (la differenza fra la durata massima prevista

e la fase alta più breve) e il limite per quel ciclo scende a $L_{APB} \leq 12{,}3$ cicli in NTSC (12,9 in PAL). Il ciclo che segue una scrittura è la lettura del codice operativo dell'istruzione successiva: è servito dall'ESP32 solo se il programma esegue codice dalla ROM PBI o dalla finestra RAMbo *e* quel codice scrive il latch o PORTB, combinazione che la ROM del modulo non usa.

6.4 Ciclo di lettura servito dalla VERA

Per i registri `$D100`–`$D11F` l'ESP32 asserisce DEV_SEL_N (17 cicli più una lettura APB dopo il campionamento); la VERA porta il dato sul bus quando DEV_SEL_N e $\overline{\text{mRE}}$ sono entrambi attivi, attraverso U19 abilitato dallo stesso DEV_SEL_N. Il margine nel caso peggiore è di 141 ns con latenza nominale e 108 ns con latenza pessimistica: le letture dei registri VERA non sono critiche in nessuno scenario (il limite su L_{APB} supera i 40 cicli).

6.5 Cicli di scrittura

6.5.1 Campionamento del dato

Il campione è preso a tempo fisso dopo il rilevamento della salita di ϕ_2. Il caso più anticipato cade 19 ns dopo l'istante in cui il dato è garantito valido (t_{MDS} più U19); il caso più ritardato deve precedere la fine del dato (t_{HW} dopo la discesa): il margine è 86 ns, 61 ns e 28 ns nei tre scenari in NTSC. **Il campionamento delle scritture è corretto in tutti gli scenari.** La figura 6.2 mostra il ciclo.

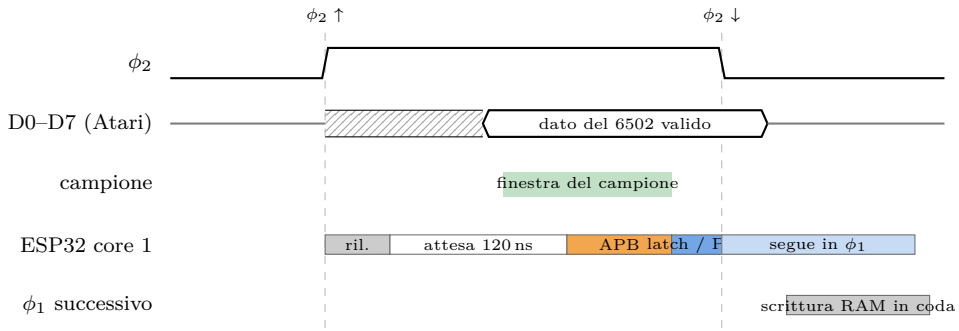


Figura 6.2: Ciclo di scrittura (NTSC, $L_{APB} = 12$, caso peggiore). Il dato è campionato nella finestra verde, interamente dentro l'intervallo di validità; gli effetti sui registri occupano il resto di ϕ_2 e, se servono, l'inizio del ϕ_1 successivo, dove si esegue anche la scrittura in memoria rimasta in coda.

6.5.2 Hold del dato verso la VERA

La VERA cattura il dato di scrittura sul fronte di discesa di `bus_write = !cs_n && !wr_n`, cioè quando $\overline{\text{mWE}}$ risale dopo la discesa di ϕ_2 [15]. Lo stesso fronte di ϕ_2 disabilita U19 attraverso il circuito di abilitazione, e da quel momento il dato al pin della FPGA non è più pilotato. Con i ritardi della tabella 3.3 il margine di hold è negativo di alcuni nanosecondi nel caso tipico e di qualche decina nel caso peggiore (figura 6.3). Il problema non dipende dal firmware ed è una delle questioni aperte (sezione 8.3).

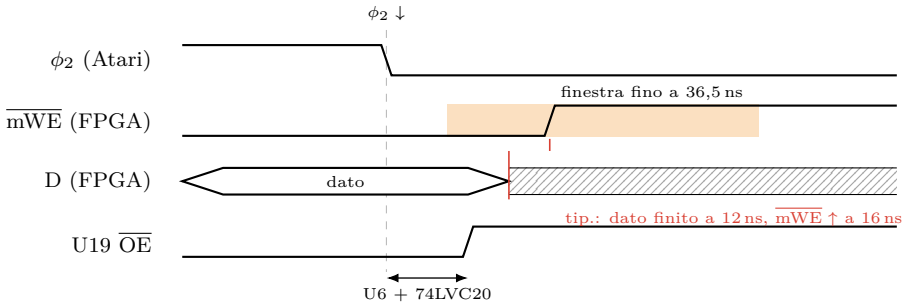


Figura 6.3: Hold del dato di scrittura alla FPGA (scala: 1 ns = 1,35 mm). Il dato smette di essere pilotato quando U19 si disabilita, mentre la VERA lo cattura alla salita di $\overline{\text{mWE}}$.

6.6 Asserzione di EXTSEL e MPD

EXTSEL e MPD arrivano sul bus Atari 34 ns dopo la salita di ϕ_2 nel caso peggiore con latenza nominale (8 ns nel caso tipico, 9 ns e −17 ns con latenza ottimistica). Un decodificatore combinatorio li asserirebbe circa 120 ns prima della salita: il ritardo residuo è di poche decine di nanosecondi nel caso peggiore e nullo o negativo nel caso tipico. Se la RAM interna attiva CAS prima di questo istante, in lettura c'è una breve contesa e in scrittura nella finestra RAMbo la scrittura raggiunge anche la RAM di base. La posizione di RAS/CAS dipende dal modello di computer e va misurata (sezione 9.2).

6.7 Interruzioni

Il core 1 esegue il ciclo principale con le interruzioni mascherate e non esegue altri task: nessun ciclo di bus viene perso per il tick di FreeRTOS o per interruzioni di periferica. Il ciclo non chiama funzioni, quindi non subisce eccezioni di finestra di registri né mancate nella cache delle istruzioni.

6.8 Sintesi grafica

Le figure 6.4 e 6.5 riportano i margini in funzione di L_{APB}; per confronto è riportato il firmware iniziale (capitolo 7).

Tabella 6.2: Esito della verifica temporale del progetto.

Percorso	Esito	Nota
Lettura ROM/RAM PBI e RAMbo	**conforme** se $L_{APB} \leq 14{,}1$	da misurare; +27 ns a $L_{APB} = 12$
Lettura dopo scrittura su latch/PORTB	**conforme** se $L_{APB} \leq 12{,}3$	sequenza non usata dalla ROM del modulo
Lettura registri VERA	**conforme**	margine ≥ 108 ns
Campionamento scritture	**conforme**	in tutti gli scenari
Contesa sul bus dati	**risolta**	circuito di abilitazione di U19
EXTSEL/MPD	**da misurare**	intorno alla salita di ϕ_2
Interruzioni	**risolto**	core 1 senza interruzioni
Hold scrittura VERA	**aperto**	sezione 8.3

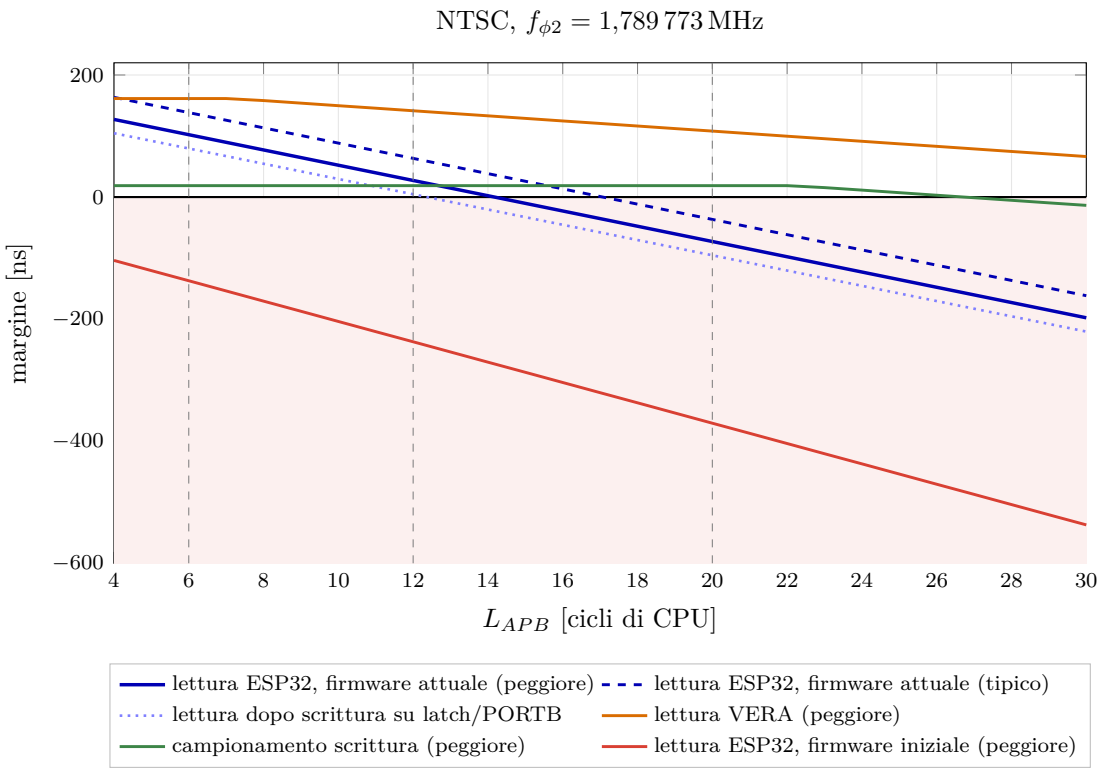


Figura 6.4: Margine dei percorsi in funzione della latenza APB (NTSC).

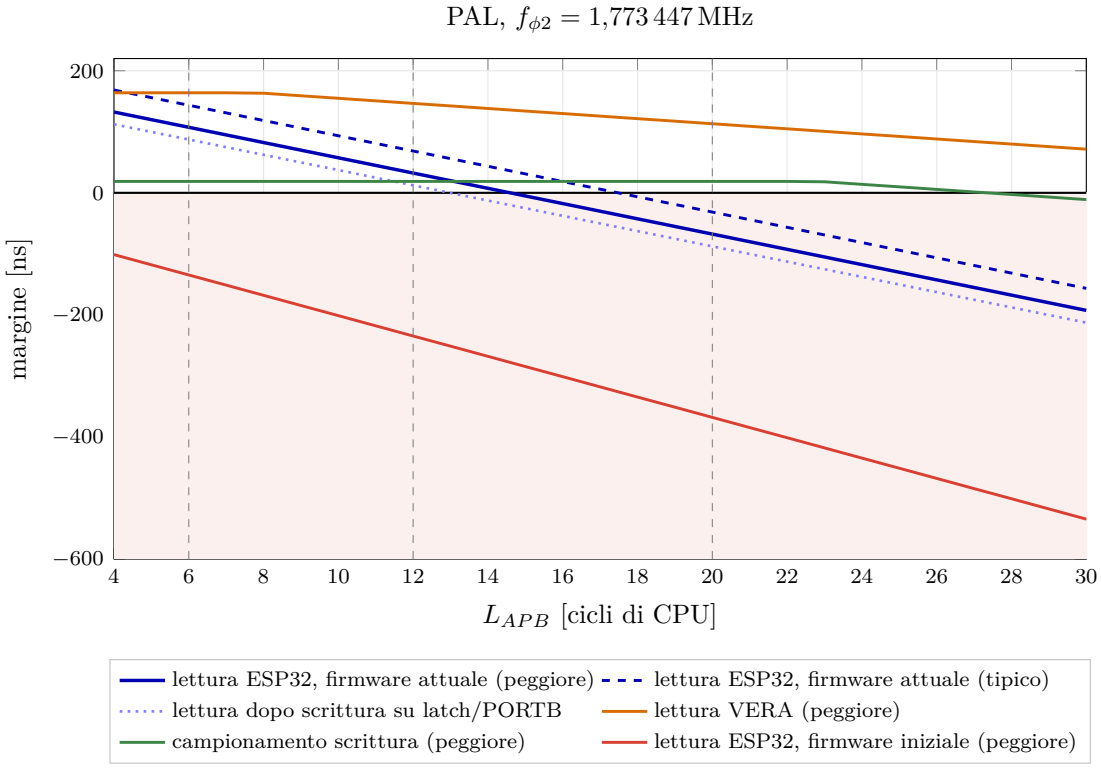


Figura 6.5: Margine dei percorsi in funzione della latenza APB (PAL).

Capitolo 7

Dalla prima versione al progetto attuale

Questo capitolo riassume l'analisi della prima versione della scheda e del firmware (commit `f311311`) e le correzioni che ne sono seguite.

7.1 Il ciclo principale iniziale

Il firmware iniziale eseguiva il ciclo principale sul core 0, con le interruzioni abilitate, e iniziava a lavorare alla salita di ϕ_2: due letture APB (`GPIO.in` e `GPIO.in1`), 20 cicli fino ai segnali di controllo e altri 27 cicli più due scritture APB per il dato, separate da barriere `memw`. Anche con accessi alle periferiche istantanei i soli 47 cicli di elaborazione (196 ns) portavano il dato oltre il limite. La figura 7.1 mostra il ciclo nel caso tipico con la latenza ottimistica, la tabella 7.1 il confronto con il firmware attuale.

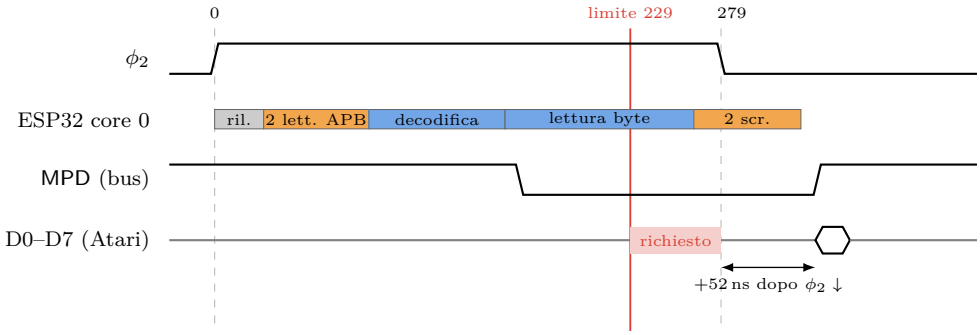


Figura 7.1: Lettura della ROM PBI nel firmware iniziale, caso tipico con $L_{APB} = 6$ cicli: il dato arriva sul bus 331 ns dopo la salita di ϕ_2, quando il 6502 ha già campionato il bus.

7.2 Problemi individuati e correzioni

Due di questi problemi meritano un commento. La sovrapposizione della RAMbo con la memoria dati non era segnalata dal linker: la sezione creata dall'attributo nel sorgente veniva collocata dopo il simbolo `_iram_end`, che ESP-IDF usa per riservare sul bus dati lo spazio occupato dal codice in IRAM, e le due viste della stessa SRAM sono regioni di memoria distinte per il linker. L'immagine della ROM, invece, era generata correttamente

Tabella 7.1: Margine del dato in lettura servita dall'ESP32 (NTSC): firmware iniziale e attuale.

	Firmware iniziale		**Firmware attuale**	
Scenario	tipico	peggiore	tipico	peggiore
ottimistico	−101	−137	138	102
nominale	−201	−237	63	27
pessimistico	−335	−370	−37	−73

Margini in nanosecondi; un valore negativo è una violazione.

Tabella 7.2: Problemi della prima versione e soluzioni adottate.

Problema	**Soluzione**
Lavoro iniziato alla salita di ϕ_2; dato oltre il limite in ogni scenario	decodifica durante ϕ_1, percorso critico in assembly (cap. 5)
Due letture APB per campionare l'indirizzo	ϕ_2, $R/\overline{W}$, A10–A15 sui canali Dedicated GPIO: una sola lettura
EXTSEL/MPD asseriti 150–300 ns dopo la salita di ϕ_2	asseriti intorno alla salita (sezione 6.6)
U19 abilitato in ogni ciclo: contesa sul bus dati in ogni lettura interna	circuito di abilitazione con un 74LVC20 (sezione 3.2)
Tick di FreeRTOS sul core del bus: 3–9 cicli persi ogni millisecondo	ciclo sul core 1 a interruzioni mascherate, log sul core 0
`bus_idx_to_page()` non espansa, eseguita da flash a ogni scrittura in `$D1xx`/`$D3xx`/`$D5xx`	pagina usata direttamente come indice; nessuna chiamata nel ciclo
ROM PBI e RAMbo in IRAM, non accessibile a byte: eccezione al primo accesso	ROM, RAM e RAMbo in DRAM
Sezione `.iram0.bss` non riservata sul bus dati: i 256 KB della RAMbo sovrapposti a `.data`, `.bss` e heap	RAMbo come `.bss` ordinaria
Immagine della ROM PBI non aggiornata (4 KB, versione precedente del driver)	immagine generata dal makefile a ogni compilazione, dimensione verificata
ROM di autotest ignorata: contesa a `$5000`–`$57FF`	stato «autotest visibile» nelle tabelle
DDR di PORTB ignorato; indirizzi speculari della PIA ignorati	pin calcolati come uscita | $\overline{\text{DDR}}$; confronto su A0–A1
Abilitazione della RAMbo da un pin non presente sulla scheda	RAMbo sempre attiva

dal makefile ma in un file diverso da quello incluso dal firmware: la ROM servita all'Atari non era quella del driver corrente.

Capitolo 8

Questioni aperte

8.1 Latenza degli accessi alle periferiche

Il limite di conformità dipende da L_{APB}, che va misurata. Se risultasse superiore a circa 14,1 cicli, il firmware non avrebbe margini residui da recuperare: il percorso critico è già ridotto al minimo e gli accessi APB per ciclo sono tre (una lettura e due scritture). La strada sarebbe allora l'architettura alternativa della sezione 8.6.

8.2 Uscite dati sempre abilitate

Con il circuito di abilitazione U19 isola l'ESP32 quando la scheda non risponde. Il firmware potrebbe quindi tenere abilitate le uscite dati anche durante ϕ_1 e spegnerle solo prima di un ciclo di scrittura o di una lettura dei registri VERA, togliendo dal percorso critico la scrittura di `GPIO.enable_w1ts`. Il dato ne trarrebbe un margine molto maggiore (limite di circa 21,7 cicli), ma in un ciclo di scrittura le uscite vanno spente prima che U19 inizi a pilotare il lato scheda, pochi nanosecondi dopo la salita di ϕ_2, e il firmware sa che il ciclo è una scrittura solo al campionamento dell'indirizzo. Questo vincolo si rompe a $L_{APB} \approx 13{,}4$ cicli: nel caso peggiore l'opzione non estende il limite attuale e ne migliora solo il caso tipico. Una violazione di pochi nanosecondi produrrebbe una breve contesa fra due uscite a 3,3 V sulla scheda, senza effetti sul dato letto dalla CPU, e si renderebbe innocua con resistenze in serie da 33–47 Ω sulle linee D0–D7 dell'ESP32.

8.3 Hold della scrittura verso la VERA

Il margine di hold della sezione 6.5.2 è negativo. Le correzioni possibili sono sostituire il 74HC00 che genera $\overline{\text{mWE}}$ con un 74LVC00, più veloce a 3,3 V, e ritardare la disabilitazione di U19 rispetto alla salita di $\overline{\text{mWE}}$, per esempio con un elemento di ritardo sul percorso di $\overline{\text{OE}}$ nei soli cicli di scrittura [15, §2.2].

8.4 Interruzioni della VERA

L'uscita IRQ della VERA è push-pull e non può pilotare direttamente la linea IRQ condivisa dell'Atari, che è open-collector: serve uno stadio open-drain (74LVC1G07) [15, §2.3].

8.5 Strapping dell'ESP32

A14 e A15 sono collegati a GPIO 45 e 46, due pin di strapping. Con GPIO 45 alto al power-on la flash interna verrebbe alimentata a 1,8 V e il chip non si avvierebbe. Il livello di A14 al momento del campionamento va verificato; in alternativa si può fissare la tensione con gli eFuse `VDD_SPI_FORCE` e `VDD_SPI_TIEH`.

8.6 Architettura alternativa: logica di bus in FPGA

Il repository contiene anche un'implementazione dello stesso controllo del bus in VHDL per una FPGA iCE40UP5K di interfaccia (`C6502_Interface.vhd`), con ROM e RAM PBI in BRAM e la RAMbo in una SRAM asincrona da 55 ns. In logica combinatoria la decodifica segue l'indirizzo di poche decine di nanosecondi, EXTSEL e MPD sono validi durante ϕ_1 e il dato arriva sul bus circa 30–40 ns dopo la salita di ϕ_2, con un margine di quasi 200 ns. All'ESP32 resterebbero l'avvio, il reset e la diagnostica.

Capitolo 9

Piano di verifica sperimentale

9.1 Misura della latenza APB

La latenza di `GPIO.in` si misura su una scheda ESP32-S3 qualsiasi, senza Atari, con il contatore di cicli della CPU (listato 9.1); la stessa struttura con `GPIO.out` misura le scritture, e ripetuta con due store consecutivi senza `memw` mostra se il bus delle periferiche le sovrappone.

```
#include <Arduino.h>
#include <soc/gpio_struct.h>
#include <xtensa/hal.h>

static void IRAM_ATTR bench(void)
{
    uint32_t sink = 0;
    portDISABLE_INTERRUPTS();
    uint32_t t0 = xthal_get_ccount();
    for (int i = 0; i < 1000; i++)
        sink += GPIO.in;
    uint32_t t1 = xthal_get_ccount();
    portENABLE_INTERRUPTS();
    Serial.printf("GPIO.in: %.2f cicli (sink %u)\n", (t1 - t0) / 1000.0f, sink);
}

void setup() { Serial.begin(115200); delay(500); bench(); }
void loop()  {}
```

Listing 9.1: Misura della latenza di lettura di `GPIO.in`.

Il valore misurato, meno i 3–4 cicli di overhead dell'iterazione, sostituisce `SCENARIOS` in `timing_model.py`; ricompilando la tesi si aggiornano tabelle e grafici.

9.2 Misure sul bus con analizzatore logico

Con l'analizzatore logico (almeno 200 MS/s) e la scheda collegata all'Atari:

1. duty cycle di ϕ_2 sul connettore, in PAL e NTSC, per confermare `BUS_PHI2_HIGH_MAX_NS`;
2. t_{ADS} reale: ritardo fra la discesa di ϕ_2 e la stabilità di A0–A15, per ridurre eventualmente `BUS_ADDR_SETTLE_NS`;
3. ritardo fra la salita di ϕ_2 e DEV_SEL_N, EXTSEL, MPD e $\overline{\text{OE}}$ di U19;

4. ritardo fra la salita di ϕ_2 e il dato sul connettore in una lettura di `$D803`, confrontato con la discesa di ϕ_2;
5. assenza di contesa: in una lettura della RAM interna il bus dati non deve essere pilotato da U19;
6. posizione di RAS/CAS della RAM interna rispetto a ϕ_2;
7. hold del dato al pin della FPGA rispetto alla salita di $\overline{\text{mWE}}$;
8. continuità del servizio: un GPIO libero (47 o 48) commutato a ogni iterazione del ciclo principale rende visibile qualunque pausa.

9.3 Prove funzionali

I programmi di prova del progetto [15] verificano il comportamento complessivo: riconoscimento della ROM PBI all'avvio, rilevamento della VERA, `TESTFX.COM` (atteso 36 PASS), una prova della RAMbo che scrive e rilegge tutti i 16 banchi controllando che la memoria di base a `$4000`–`$7FFF` resti invariata, e `TESTIRQ.COM` dopo il montaggio dello stadio open-drain.

Capitolo 10

Conclusioni

Il modulo Vera-Module-RBL-XE affida a un ESP32-S3 l'intera decodifica del bus di un Atari XL/XE: registri della VERA, dispositivo PBI ed emulazione della RAMbo 320K. L'analisi della prima versione ha mostrato che l'impostazione era corretta nella logica ma non nei tempi: il lavoro iniziava alla salita di ϕ_2 e i cicli di lettura serviti dal microcontrollore non potevano rispettare il setup del 6502, il bus dati era in contesa in quasi ogni lettura e il firmware conteneva difetti di collocazione in memoria che lo avrebbero bloccato.

Il progetto attuale sposta la decodifica nella fase ϕ_1, quando l'indirizzo è già stabile, riduce il campionamento a un solo accesso alle periferiche grazie ai canali Dedicated GPIO, scrive il percorso critico in assembly e lo esegue su un core dedicato senza interruzioni; il circuito di abilitazione di U19, realizzato con un solo 74LVC20, elimina la contesa sul bus dati. Secondo il modello il dato rispetta il setup del processore nel caso peggiore fino a una latenza APB di circa 14,1 cicli in NTSC e 14,5 in PAL, il campionamento delle scritture è corretto in tutti gli scenari e i segnali EXTSEL e MPD arrivano intorno al fronte di salita di ϕ_2. La differenza fra PAL e NTSC resta di pochi nanosecondi.

Il risultato è analitico e dipende da una grandezza non documentata, la latenza degli accessi alle periferiche: misurarla è il primo passo del piano sperimentale. Restano aperti l'hold della scrittura verso la VERA e lo stadio open-drain per le interruzioni. Se la latenza risultasse troppo alta, la logica di bus in FPGA già progettata offre un'alternativa con un margine ampio, lasciando all'ESP32 i compiti non critici.

Bibliografia

[1] A. Lee, *Altirra Hardware Reference Manual*, versione 07/05/15, 2015.

[2] Rockwell International, *R6500 Microprocessors (CPU) – R6502, R6502A*, data sheet, caratteristiche AC dei cicli di lettura e scrittura.

[3] MOS Technology, *MCS6500 Microcomputer Family Hardware Manual*, 1976.

[4] Espressif Systems, *ESP32-S3 Series Datasheet*, v2.2.

[5] Espressif Systems, *ESP32-S3 Technical Reference Manual*, capitoli «System and Memory», «IO MUX and GPIO Matrix» e «Dedicated GPIO».

[6] Espressif Systems, *ESP-IDF Programming Guide v4.4*: «Dedicated GPIO» e «Memory Types»; `soc/esp32s3/include/soc/soc.h`.

[7] NXP Semiconductors, *74LVC4245A – Octal dual supply translating transceiver; 3-state*, data sheet.

[8] Nexperia, *74LVC20A – Dual 4-input NAND gate*, data sheet.

[9] Texas Instruments, *SN74HCT04 Hex Inverters*, data sheet.

[10] Texas Instruments, *SN74HC00 Quadruple 2-Input Positive-NAND Gates*, data sheet.

[11] Lattice Semiconductor, *iCE40 UltraPlus Family Data Sheet*.

[12] F. van den Hoef, *VERA – Versatile Embedded Retro Adapter*, HDL 48.0.1, progetto Commander X16.

[13] ICD Inc., *RAMBO XL Memory Upgrade – manuale*.

[14] Progetto Vera-Module-RBL-XE, `PIN-MAPPING.md` e `firmware/MCU/README.md`.

[15] Progetto Vera-Module-RBL-XE, `VERA-ATARI-HW-REQUIREMENTS.md`.

Appendice A

Il modello temporale

Lo script `timing_model.py` contiene i parametri delle tabelle 2.2, 3.3 e 5.1 e calcola i percorsi descritti nel capitolo 6, sia per il firmware attuale sia per quello iniziale. Produce `data/numbers.tex`, con una macro LaTeX per ogni valore citato nel testo, e i file `data/slack_ntsc.dat` e `data/slack_pal.dat` per i grafici. Per aggiornare la tesi dopo una misura:

```
python3 timing_model.py      # rigenera data/
pdflatex tesi.tex            # due volte, per i riferimenti
```